

CLIMB at VBS 2027: Multi-Signal Keyframe Retrieval with Collaborative Ad-Hoc Search

Tobias Wurzer¹[0009-0002-0965-6877], Sebastian Eisner¹, and Klaus Schöffmann¹[0000-0002-9218-1704]

Univeristy of Klagenfurt, 9020 Klagenfurt, Austria

Abstract. We present CLIMB, an interactive video retrieval system built for the Video Browser Showdown 2027. Since a single visual embedding is vague in places where a task hint is precise, a sign or text on screen, or a discussion about a specific topic, CLIMB represents every shot as a set of keyframes and retrieves over four independent signals. Visual embeddings, on-screen text, generated captions, and speech transcripts are fused by rank rather than score. The choice between lexical and semantic matching was made per signal by measurements. Queries can be chained into temporal sequences, in which each stage is a full query in its own.

For Ad-hoc video search, CLIMB treats a task as a single shared team activity, hiding the shots a teammate has submitted from everybody else's results, while search, browsing, and even submission continue to work, even when the shared service is unreachable.

Keywords: Video Browser Showdown · Interactive Video Retrieval · Collaborative Search · Semantic Embedding.

1 Introduction

The Video Browser Showdown [9] asks interactive video retrieval systems to solve known-item, ad-hoc, and question-answering tasks against video collections that nobody can watch through. For VBS 2027, the main collection is V3C [10] with 28,450 videos, roughly 3,800 hours, and about 4.7 TB, segmented into 4.14 million master shots, alongside the smaller MVK [12] and GynSurgLHE [6] collections.

Two problems follow from that scale. The first is that a single visual embedding is vague when it comes to the places a task hint might be precise. Textual hints can name signs seen on screen, and even in visual questions, those are prominent hints to search for. The second is that AVS scoring rewards distinct correct shots, so a team of n operators searching independently spends a substantial share of its submissions re-finding what a teammate has already sent.

CLIMB addresses both. It indexes every video as scenes and keyframes, retrieves over four independent signals, fuses them by rank, and treats an AVS task as a single shared activity rather than n parallel ones, vanishing duplicate scenes from everybody's list. Concretely, we contribute:

1. Keyframe-level retrieval fusing four heterogeneous signals by rank, with the choice between lexical and semantic matching made per signal by measurement (Sect. 4.1).
2. A temporal query language, $A \gg B \gg (120) C$, in which every stage is itself a full query (Sect. 4.3).
3. A collaborative AVS mode built around the scoring rule, which degrades safely when the shared service is unreachable (Sect. 6).
4. An index engineered for everyday hardware: binary-quantized HNSW with oversample-and-rerank, fed by a resumable ingest pipeline (Sect. 3).

2 System Overview

CLIMB is built from six components (Fig. 1). Decoding each video, segmenting it into scenes, selecting the keyframes, computing the embeddings, reading the text on screen, as well as captioning and transcribing, happens in an offline pipeline written in Python and described in Sect. 3. The resulting scenes, keyframes, and four indexes that get queried by the retrievers land in a PostgreSQL database.

At query time the retrieval stack and main application run as separate processes, due to the simple reason that they are written in different languages. While the models and the fusion require a Python environment, media serving, DRES communication, and the AVS mirror are based on Node.js and Express. Both processes connect to the database: the backend reads the scene and keyframe metadata directly and only delegates embedding, ranking, and fusion to the search engine. The React frontend (Sect. 5), however, only talks to the backend. An unavailable search or collaboration service still allows for browsing and submission.

At VBS we intend to run one complete instance of CLIMB per operator so that keyframes and video playback are served from localhost rather than over wifi. Only the AVS session service (Sect. 6) would be shared between operators, and the state it exchanges is a few tens of kilobytes of JSON per session, polled every four seconds. Under this deployment, the only data leaving an operator’s machine would be single-digit kilobytes per second.

3 Indexing Pipeline

3.1 Scenes and Keyframe Selection

The V3C dataset provides scene boundaries. These scene boundaries are the center of our encoding. They get used to splitting the video into the different scenes. For each of these scenes, we select between 2 and 32 keyframes according to the length of the scene. One keyframe per four seconds according to the following formula:

$$k(s \in S) = \min(\max(\text{ceil}(t(s)/4), 2), 32), \quad (1)$$

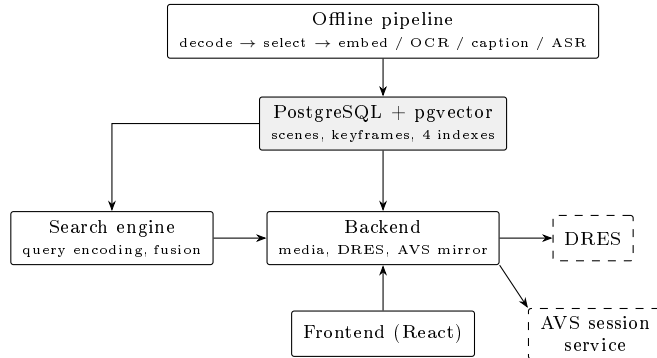


Fig. 1. CLIMB architecture. Every component shown runs locally on each machine, except for the two external services: DRES and the AVS session service, which is deployed once.

where S is the set of scenes in a video, and $t(s)$ is the duration of the scene s in seconds.

We start by selecting the most typical frame and then repeatedly add the frame, which is the most different from the selected keyframes so far, which selects a broad coverage of keyframes in a scene and not near duplicates. Afterward we reject blank frames. A blank frame is a frame that is nearly flat and has almost no edges. This reduced the number of keyframes sampled from our subset from 99,661 at 1 fps to 35,558 keyframes.

Damaged bitstreams are indexed from what we were able to decode, flagged, and logged.

3.2 Feature Extraction

CLIMB extracts four signals per scene. Those were chosen so that each one covers a failure of the others. Where an embedding is vague, a sign or a spoken name is precise. Where there is no visual hint, there might be a spoken one, and where there is no exact text known to search for, there might be a general description of the scene to search for.

Whilst we have four different signals, they are deliberately not applied equally. OCR reads every keyframe, but captioning is only done once per scene. The center of the application is SigLIP2, which embeds each keyframe into a 1024-dimensional vector space and handles anything visual [13]. OCR also looks at every keyframe, extracting the text on screen. The model used is PP-OCR [2] via PaddleOCR where available and rapidocr-onnxruntime otherwise.

The caption model only captions the first keyframe of each scene, since it is expensive to caption an image and the caption barely changes over the span of a scene. This results in just 14,345 captions in comparison to 35,558 OCR passes and SigLIP2 embeddings. The model that we intend to use for the large dataset

Table 1. The four retrieval signals. Score ranges are the top 20 values over 20 queries on the V3C1-200 dataset we used for testing; the first three are cosine similarities, OCR is a lexical cover-density rank. They are *not* comparable across rows, which is why fusion is by rank (Sect. 4.1).

Signal	Unit	Matched by	Score range
SigLIP2 embedding	keyframe	cross-modal embedding	0.084–0.174
On-screen text (OCR)	keyframe	lexical matching	0.091–0.167
Scene caption	scene	text embedding	0.809–0.931
Transcript (ASR)	video $\rightarrow$ scene	text embedding	0.762–0.859

will be Qwen2-VL-2B-Instruct [15], but for testing purposes we used a smaller model (SmolVLM-256M-Instruct [5]) for a first draft.

Additionally, we use Whisper [7] as an ASR model, which transcribes what was said during the video and gets later mapped to scenes via overlapping the timestamps of the transcript with the timestamps of the scenes.

In order to encode the textual descriptions of the caption model as well as the transcripts into a semantic embedding, we use multilingual-e5-base [14], which embeds the texts into a 768-dimensional vector space.

3.3 Scaling to the Full Collection

The pipeline never requires the collection to be local. Videos are fetched in batches, processed, and released again by `purge` as soon as the database shows an artifact is spent, so the working set stays bounded no matter how much storage is at hand. An `ingest_jobs` table records each video’s stage, and workers claim batches with `FOR UPDATE SKIP LOCKED`, so a decode on one machine and an embedding pass on a GPU elsewhere share one queue with no coordinator. Every GPU stage also takes `--shard N --shards M`. Each stage derives its outstanding work from the database, so an interrupted run is simply re-run. The index is HNSW [4] over binary-quantized embeddings, reranked against full-precision vectors: 5.3 GB at 12.4M keyframes where half precision would need 28 GB.

4 Retrieval Model

4.1 Rank-Based Fusion at Keyframe Level

As we discussed before, we use four different encoders to encode as broad a spectrum of information as possible. Each signal then returns a list of ranked objects, scenes, or keyframes, respectively. Whilst the visual signal and OCR signal natively return a list of ranked keyframes, we only captioned the first keyframe of each scene. Therefore, its signal is scene-based. In order to not inflate the rank of the first keyframe of each scene, this result is then shared with all keyframes of the scene, meaning all keyframes of a scene score the same

amount of points for the caption-encoded signal. For the audio signal, it encodes scenes via segments. Each segment gets a den mapped to scenes, and the rank of a scene gets expanded to the keyframes, just as with the caption encoder.

The keyframes are then merged via weighted Reciprocal Rank Fusion (RRF) [1]. Which works just as RRF, but the signals get weighted depending on their source, allowing us to highlight keyframes found via specific encoders, i.e., OCR, where a lexical hit is much more likely to belong to the correct scene than a similar visual embedding. The updated scoring formula is therefore:

$$RRFscore(d \in D) = \sum_{r \in R} \frac{w(r)}{k + r(d)} \quad (2)$$

where D is the set of keyframes, R is the set of encoders, $w(r)$ is the weight of a specific encoder signal, k is a constant set to 60, and $r(d)$ is the rank the encoder r gave the keyframe d [1]. The constant k was chosen by the authors of the original RRF paper and helps to reduce the influence of outliers.

We use the rank to combine the different encoders, since the four signals do not produce comparable scores on their own. A small 20-query test on our subset shows that the best visual result is a score of 0.17, whilst the best caption match was a 0.93 (Table 1). This vast difference in scores shows that in order to map them to a common scale, we would need to calibrate it per retriever, and also update that every time we decide to change a model. Choosing a ranked based fusion approach, we can skip this sensitiv calibration entirely, whilst still allowing for minor tweaking using the assigned weights.

It's important to note that the visual number is that low, since the embedding is cross-modal. SigLIP2 embeds the text and image into a shared vector space, leading to a modality gap, meaning a score of 0.15 is a strong match, not a weak one [3].

4.2 Semantic and Lexical matching

The visual matching using SigLIP2 is semantic since we want to compare text and images. The other three signals compare text with text, so a lexical match would be possible. But lexical matching on captions doesn't work. If you search for "law enforcement," for example, the correct scene has a description, "[...] One person is wearing a police uniform." Not a single word matches. And that goes not only for that scene; "law" and "enforcement" don't occur in a single caption over the whole 25 hours, with 14.345 captions we extracted until now. So a lexical matching would not only return an incorrect list but an empty set of not a single scene.

The audio also gets matched semantically to allow for similar matching and also cross-lingual matching. One thing to note, however, is that the user has the ability to do a lexical search on spoken words using the "said:" operator, allowing them to search for scenes where a specific said text was given in the task description.

Table 2. Query syntax: a plain query searches all four signals; an operator prefix restricts it to one. >> orders two to four queries into a chain within a time window, confined to a single video.

Input	Effect
a drawing of people on a beach	all four signals search
text: state capitol	on-screen text only
said: new york	spoken words only
-video:00191	exclude a video (repeatable)
a video game sunset >> door	one scene, then the other (same video)
a video game sunset >>(10) door	similar to A >> B but, within 10 seconds

OCR is the opposite of audio; the entire point of OCR is to find the keyframe, showing a text noted in the task description and not something similar; for that, we have a visual embedding.

4.3 Query Language and Temporal Chains

Sometimes one does not only want to describe only a single scene but a sequence of them, first a bridge, then a beach. A plain text query searches all four signals at once, and the operator prefixes in Table 2 allow it to narrow it down to a single source, whilst >> joins queries into ordered temporal chains.

The chaining makes the prefixes much more powerful. A query **A >> B >> C** is 3 stages with two independent windows, where each stage is a complete query on its own. **text: pegasus >> mountain bike race** finds a scene where first a text that reads "pegasus" is searched, followed by a scene that matches with mountain bike race, independent of a single encoder. An unannotated chain searches for scenes in the same video that are at maximum 60 seconds apart, annotated with (120) or (d120); it's 120 seconds, and (d500ms) is half a second. The maximum delta is ten minutes, and the maximum amount of stags in a chain is four.

5 Interactive Search

5.1 The Search Loop

A search typically follows a simple loop. An operator types a query and gets a resulting grid of keyframes. When clicking on a thumbnail, a five-second scene preview gets displayed, starting at the selected keyframe. A bit under the video player, there is the filmstrip, where one can see all the keyframes and navigate to them efficiently. Keyframe times get filled in automatically but are editable, so they can be changed to the exact submitted frame. Additionally, there is a variety of buttons helping to either further narrow down the search or submit the selected frame.

The large red *Submit to DRES* button does the latter and submits the selected keyframe/time to the server. If one realizes that the selected keyframe

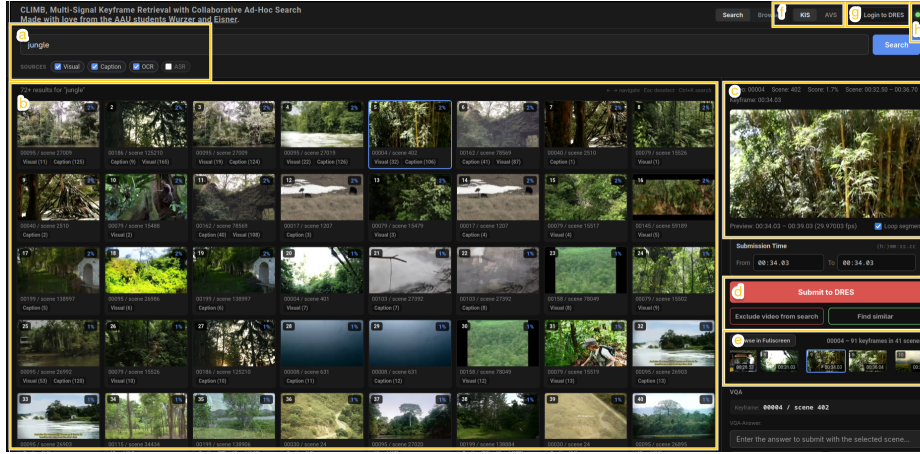


Fig. 2. The CLIMB interface: (a) query bar and source filters, (b) the scene result grid, (c) detail panel with player, (d) submission and search refinement buttons, (e) keyframe filmstrip, (f) AVS session tab, (g) DRES connection controls and (h) health status overview.

does not belong to the correct video, the whole video can be excluded from the search using the *Exclude video from search* button, and the *Find Similar* button searches for keyframes with a similar embedding to the one selected. Useful if the keyframe is good but hard to describe why it is so good.

Further down, there is the VQA segment, where an optional Visual Question Answering task can be answered and the answer submitted.

To speed up the search, the queried pages are cached. The first page runs the search, and later pages are loaded from the cache on scrolling. This pagination removes the need for rephrasing the query if the desired video is not found in the first few results, where the caching reduces the cost of loading these pages.

The "exclude" and "find similar" buttons described above, as well as a new query, search for a new set of results that replaces the grid of keyframes shown, further refining the search and closing the search loop.

5.2 Submitting to DRES

As with every browser at VBS, the DRES-Submission pipeline is an integral part of CLIMB [8, 11]. On the top right, a login popup can be opened, where the *server URL*, *evaluation name*, and *credentials* can be entered. Important to note is that CLIMB falls back to the first available evaluation when the one entered doesn't exist and clearly shows this to the operator.

Every submission returns a status, containing a status code, summary, and verdict. This gets displayed to the user after the *Submit to DRES* button, when available, in green when the submission was correct, yellow when there is no verdict yet, and red when it was wrong or rejected. In addition to the default

KIS search loop described in Sect. 5.1, the user can submit a text or a shot and text answer to a VQA task.

An AVS submission follows a bit of a different route, described in Sect. 6, but from a user perspective it follows the same principle as a default KIS task, once switched to the AVS-Tab on the top right of the UI, and a session is joined. The only difference is that submissions with the result *correct* and *accepted-awaiting-verdict* get hidden from everyone.

6 Collaborative Ad-Hoc Video Search

Since AVS scoring rewards distinct correct shots, and a team of N people searching independently will find the same obvious results first, leading to a large fraction of duplicate submissions.

We solved this problem by making an Ad-Hoc Video Search (AVS) session a shared state. This allows us to hide submitted scenes from everyone’s results. Covered videos get dimmed and marked as covered, but not hidden to promote a diverse submission set, but not totally hidden to allow for submissions of multiple scenes inside a single video. An AVS session is a random 4-letter code, whereby each member can create a new session or join one. Sessions expire after 2 hours of inactivity, more than long enough to outlive a task block but short enough to not excessively use the server’s memory.

It’s important to note that the search never depends on the AVS collaboration. A slow session service or missing service session can never slow a search down, and submissions also never depend on it. We first send out the POST request to the DRES server before even trying to reach the collaboration service.

We chose to extract this into an additional service as a separate deployable, independent of the remaining architecture. This allows for everyone on the team to run a local copy of CLIMB while still being able to cooperate, no central server necessary.

7 Preliminary Results

The pipeline is implemented for the full V3C collection; the numbers below were measured on the 200-video subset available at the time of writing (25,h, 14,345 shots, 35,558 keyframes).

Scene-adaptive selection produces those 35,558 keyframes against 99,661 for uniform 1 fps sampling and still covers every shot.

Against exhaustive exact search over 7,543 keyframes (108 ms), oversampling 1,000 candidates before the exact rerank recovers 15 of the true top 20 in 29 ms, 2,000 recovers 18 in 54 ms, and 4,000 all 20 in 91 ms. The gap is the binary quantization rather than the index: the ANN stage ranks by Hamming distance over 1-bit vectors. CLIMB defaults to 1,000 but keeps it easily configurable.

A far simpler version of CLIMB won our university’s course-internal retrieval competition preceding this submission, but we expect VBS to be considerably

harder. Query latency has been measured at 7,543 keyframes rather than the about 4M shots of the full collection, and neither MVK nor GynSurg resembles the imagery SigLIP2 was trained on.

8 Conclusion

We presented CLIMB, an interactive video retrieval system for VBS 2027. It fuses four heterogeneous signals, namely visual embeddings, on-screen text, generated captions, and transcripts, by rank at the keyframe level, with the choice between semantic and lexical matching made per signal, and it lets an operator chain several such queries into a temporal sequence.

Its collaborative mode is built around the AVS scoring rule. Shots a teammate has already submitted disappear from everyone’s results, while search, browsing, and submission continue unaffected when the shared session service cannot be reached.

Our next steps are completing the ingest of the full collection and closing the domain gap on MVK and GynSurg, where the general-purpose visual embeddings from SigLIP2 have the least to offer.

References

1. Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. p. 758–759. SIGIR '09, Association for Computing Machinery, New York, NY, USA (2009). <https://doi.org/10.1145/1571941.1572114>
2. Li, C., Liu, W., Guo, R., Yin, X., Jiang, K., Du, Y., Du, Y., Zhu, L., Lai, B., Hu, X., et al.: Pp-ocrv3: More attempts for the improvement of ultra lightweight ocr system. *arXiv preprint arXiv:2206.03001* (2022)
3. Liang, V.W., Zhang, Y., Kwon, Y., Yeung, S., Zou, J.: Mind the gap: Understanding the modality gap in multi-modal contrastive representation learning. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) *Advances in Neural Information Processing Systems*. vol. 35, pp. 17612–17625. Curran Associates, Inc. (2022). <https://doi.org/10.52202/068431-1280>, https://proceedings.neurips.cc/paper_files/paper/2022/file/702f4db7543a7432431df588d57bc7c9-Paper-Conference.pdf
4. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **42**(4), 824–836 (2020). <https://doi.org/10.1109/TPAMI.2018.2889473>
5. Marafioti, A., Zohar, O., Farré, M., Noyan, M., Bakouch, E., Cuenca, P., Zakka, C., Allal, L.B., Lozhkov, A., Tazi, N., et al.: Smolvlm: Redefining small and efficient multimodal models. *arXiv preprint arXiv:2504.05299* (2025)
6. Nasirihaghighi, S., Ghamsarian, N., Peschek, L., Munari, M., Husslein, H., Sznitman, R., Schoeffmann, K.: Gynsurg: A comprehensive gynecology laparoscopic surgery dataset. In: *Proceedings of the 33rd ACM International Conference on*

- Multimedia. p. 13141–13147. MM '25, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3746027.3758267>, <https://doi.org/10.1145/3746027.3758267>
7. Radford, A., Kim, J.W., Xu, T., Brockman, G., Mcleavey, C., Sutskever, I.: Robust speech recognition via large-scale weak supervision. In: Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., Scarlett, J. (eds.) *Proceedings of the 40th International Conference on Machine Learning*. *Proceedings of Machine Learning Research*, vol. 202, pp. 28492–28518. PMLR (23–29 Jul 2023), <https://proceedings.mlr.press/v202/radford23a.html>
 8. Rossetto, L., Gasser, R., Sauter, L., Bernstein, A., Schuldt, H.: A system for interactive multimedia retrieval evaluations. In: Lokoč, J., Skopal, T., Schoeffmann, K., Mezaris, V., Li, X., Vrochidis, S., Patras, I. (eds.) *MultiMedia Modeling*. pp. 385–390. Springer International Publishing, Cham (2021)
 9. Rossetto, L., Schoeffmann, K., Gurrin, C., Lokoc, J., Bailer, W.: Results of the 2025 video browser showdown. *ArXiv abs/2509.12000* (2025), <https://doi.org/10.48550/arXiv.2509.12000>
 10. Rossetto, L., Schuldt, H., Awad, G., Butt, A.A.: V3c – a research video collection. In: Kompatsiaris, I., Huet, B., Mezaris, V., Gurrin, C., Cheng, W.H., Vrochidis, S. (eds.) *MultiMedia Modeling*. pp. 349–360. Springer International Publishing, Cham (2019)
 11. Sauter, L., Gasser, R., Schuldt, H., Bernstein, A., Rossetto, L.: Performance evaluation in multimedia retrieval. *ACM Transactions on Multimedia Computing, Communications and Applications* (Oct 2024). <https://doi.org/10.1145/3678881>, <https://doi.org/10.1145/3678881>
 12. Truong, Q.T., Vu, T.A., Ha, T.S., Lokoč, J., Wong, Y.H., Joneja, A., Yeung, S.K.: Marine video kit: A new marine video dataset for content-based analysis and retrieval. In: Dang-Nguyen, D.T., Gurrin, C., Larson, M., Smeaton, A.F., Rudinac, S., Dao, M.S., Trattner, C., Chen, P. (eds.) *MultiMedia Modeling*. pp. 539–550. Springer International Publishing, Cham (2023)
 13. Tschannen, M., Gritsenko, A., Wang, X., Naeem, M.F., Alabdulmohsin, I.M., Parthasarathy, N., Evans, T., Beyer, L., Xia, Y., Mustafa, B., H'enaiff, O., Harmen, J., Steiner, A., Zhai, X.Q.: Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *ArXiv abs/2502.14786* (2025), <https://arxiv.org/abs/2502.14786>
 14. Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., Wei, F.: Multilingual e5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672* (2024)
 15. Wang, P., Bai, S., Tan, S., Wang, S., Fan, Z., Bai, J., Chen, K., Liu, X., Wang, J., Ge, W., et al.: Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191* (2024)